Bayyan❤■ Muhaidar

Complete Integrated Application Specification

React • TypeScript • Vite • Supabase • Installable Progressive Web App

Purpose

A private, romantic one-to-one messaging application for invited users, combining real-time chat, media sharing, voice notes, a Snapchat-style camera, filters, decorative overlays, persistent wallpapers, account recovery, and message activity indicators in one integrated project.

Project requirements document

Prepared: 25 October 2026

1. App Identity and Technology

- App name: Bayyan❤■ Muhaidar.
- Frontend: React, TypeScript, and Vite.
- Backend: Supabase PostgreSQL, Authentication, Realtime, and private Storage.
- Distribution: installable Progressive Web App (PWA), responsive on Android, iPhone, tablets, and Windows computers.
- All functions must remain part of one integrated project. Do not create a separate camera app or replace existing chat features.

2. Accounts, Login, and Security

- Support the two invited accounts: muhaidar and bayyan. Access to the private conversation and its media must be limited to the authorized accounts unless the owner deliberately invites another person.
- Username-and-password sign-in. Because Supabase Auth primarily authenticates with email or phone, implement a safe username-to-account mapping or require a verified email/username mapping; never expose account passwords in frontend code.
- Create-account flow with server-side or trusted administrative invitation controls so random visitors cannot automatically join the private chat.
- Sign-in and sign-out options.
- Forgot-password / password recovery flow using Supabase Auth email recovery or another verified recovery method.
- Use Supabase Row Level Security (RLS) policies to restrict rows to authenticated, authorized participants.
- Never place Supabase service-role keys, passwords, or privileged secrets in frontend environment variables or client code.

3. Private Chat and Messaging

- Real-time one-to-one text messaging and persistent history across devices.
- Edit sent messages and delete messages according to explicit author/participant permissions.
- Show message timestamps, sending/sent/error states, and useful retry feedback when delivery fails.
- Support image, video, and audio attachments using private Supabase Storage.
- Voice-note recording, playback, and download where the browser supports the required APIs.
- Validate upload MIME types and file sizes on the client and enforce appropriate restrictions and access policies on the backend.
- Use authorized access or time-limited signed URLs for private media; do not make private conversation files publicly readable.

4. Read Receipts, Open/Seen Status, and Typing

- Show when the other participant is typing, using a temporary Realtime presence/broadcast event; typing status should disappear when typing stops, the user leaves, or the connection ends.
- Show sent, delivered, and seen/read indicators where the implementation can reliably determine each state.
- Record a per-user read cursor or read receipt when the conversation is opened and messages are actually viewed. Avoid claiming a message was read merely because it was delivered or the app was opened in the background.
- Optional message-open status should be clearly defined: a text message is 'seen' only when the recipient has opened the conversation and the message has entered the visible/read region.

- Explain that read receipts and typing indicators require an active network connection and may be delayed by browser suspension or offline use.

5. Snapchat-Style Camera

- Live front- and rear-camera previews, camera switching, capture photo, and record video.
- Apply selected effects to the actual captured photo/video, not only to a decorative preview.
- Filter carousel: Original, Golden Hour, Warm Glow, Pink Romance, Purple Dream, Cool Blue, Vintage, Cinematic, Black and White, and High Contrast.
- Decorative animated overlays: hearts, sparkles, flowers, sunglasses, and rainbow.
- Optional face-tracking overlays using a compatible face-landmark library and required model assets.
- Timer controls, retake, preview, and send captured media directly to the chat.
- Import photos/videos from the device gallery.
- Request camera and microphone permissions with helpful, human-readable error messages.
- Use a flashlight/torch only when the device and browser expose compatible support.
- Stop camera and microphone streams when the camera screen closes or the user leaves capture mode.

6. Wallpapers and Interface

- Allow the user to choose a chat wallpaper from the device gallery and provide a default wallpaper.
- Persist wallpaper preference so it remains selected after reopening the app and, where appropriate, across devices.
- Romantic dark-plum, pink, and red visual theme.
- Responsive layouts for phones, tablets, and desktop computers.
- Bottom navigation on mobile.
- Camera screen with a large preview and horizontally scrollable filter carousel.
- Accessible button labels, visible focus states, loading indicators, and understandable error messages.

7. Required Project Files

- package.json
- index.html
- src/main.tsx
- src/App.tsx
- src/index.css
- src/lib/supabase.ts
- src/lib/chat.ts
- src/lib/camera.ts
- src/lib/filters.ts
- public/manifest.webmanifest
- public/icons/app-icon.svg
- supabase/schema.sql
- .env.example
- README.md

8. Backend and Data Design

- Use Supabase PostgreSQL for profiles/authorized participants, conversations, messages, read receipts, and any persistent preferences required by the app.
- Enable RLS on all private tables and write policies for select, insert, update, and delete operations.
- Configure Supabase Realtime for messages and presence/typing events.
- Use a private Storage bucket with policies tied to authenticated conversation participants.
- Use environment variables for the Supabase project URL and publishable/anonymous key.
- Provide schema SQL and clear setup instructions. Do not claim cloud sync works until the user configures a real Supabase project and credentials.

9. Security and Validation

- Restrict account creation to authorized invitees; do not allow arbitrary users to join the private conversation.
- Validate message length, file type, and file size before upload, and enforce server-side/storage restrictions wherever possible.
- Protect edit/delete actions with database policies, not just hidden frontend buttons.
- Use secure password reset links and redirect URLs configured in Supabase.
- Do not store plaintext passwords in application tables or local storage.
- Treat camera, microphone, gallery, and notification permissions as browser/device-controlled permissions.

10. Setup Requirements

Supabase

- Create a Supabase project and configure Authentication, database tables, RLS policies, Realtime, and a private Storage bucket.
- Set the project URL and publishable/anonymous key in a local .env file based on .env.example.
- Configure the allowed site URLs and password recovery redirect URL in Supabase Authentication settings.
- Create/invite the two intended users through the chosen secure account-provisioning flow.

Browser and device

- Camera and microphone access generally requires HTTPS in deployment, or localhost during local development.
- Users must grant camera/microphone permissions. Hardware capabilities such as torch/flashlight vary by device and browser.
- Optional face tracking needs a compatible library plus any model files/assets and licensing/configuration required by that library.
- PWA installation depends on browser support and a valid web manifest/service-worker setup.

11. Windows 11 Local Development

Install a current Node.js LTS release and Git if needed. Open PowerShell in the project folder and run:

```
npm install

Copy-Item .env.example .env

# Edit .env and enter your Supabase project URL and publishable/anon key

npm run dev
```

Open the localhost URL printed by Vite in the terminal. Keep the terminal running while testing.

Before deployment, run the project scripts provided in package.json, such as npm run build and npm run preview, and resolve any TypeScript/build errors.

12. Deployment

- Build the production app with npm run build.
- Deploy the generated dist folder to a static host that supports HTTPS, such as a suitable Vite-compatible hosting provider.
- Set the same public Supabase URL and publishable/anonymous key as deployment environment variables.
- Add the production domain to Supabase Authentication's allowed redirect/site URLs.
- Verify sign-in, password recovery, RLS, realtime messages, typing, read receipts, private file access, camera capture, and PWA installation on real devices before sharing the app.

13. Testing and Acceptance Checklist

- Both authorized users can sign in and sign out; unauthorized users cannot access private messages or files.
- Account creation/invitation and password recovery work with configured Supabase Auth.
- Messages synchronize between devices and persist after refresh.
- Edit/delete permissions are enforced by the database.
- Typing and read/seen states update correctly and do not falsely report reads.
- Private images, videos, audio, and voice notes upload, play, and remain protected.
- Camera switching, capture, filter application, video recording, retake, gallery import, and send-to-chat are tested on supported devices.
- Camera streams stop when leaving the camera screen.
- Wallpaper preference persists after reopening.
- Production build completes without TypeScript or compilation errors.

14. Delivery Status and Important Limitation

This PDF is the complete requirements and setup brief for the integrated application. It is not itself the runnable source-code project, and creating this document does not install dependencies, configure a real Supabase backend, deploy the app, or verify a compilation. Those actions require generating and testing the actual project files in a code/build workspace, plus the user's Supabase project credentials and deployment settings.

No cloud synchronization, authentication, camera recording, or read receipts should be described as working until implemented and tested against the configured project.